Generational STAR Stories: AMD GPU Performance → Architectural Influence

Framing principle: Every story must end with *"…and this data/recommendation influenced [specific architectural change] in the next generation."* That's what elevates a perf engineer story to a systems architect story.

STAR #1 — BERT/DLRM Memory-Bound Analysis → HBM Capacity Roadmap (MI200 → MI300X)

Situation. On MI100 (32GB HBM2) and MI200/MI250X (64–128GB HBM2e), training and inference for BERT-Large, DLRM, and early transformer workloads hit a recurring wall: once batch size or sequence length grew, the model had to be split across GPUs via tensor parallelism, and AllReduce traffic over Infinity Fabric became the dominant runtime cost. Customers running LLM-style workloads were asking why AMD couldn't match NVIDIA's single-GPU fit for 70B-class models.

Task. Quantify how much of BERT/DLRM wall-clock time on MI200 was spent in memory stalls vs compute vs inter-GPU comms, and produce an architectural recommendation for the next generation's memory subsystem that the CDNA architects could act on.

Action. Built a unified profiling methodology on MI200 combining `rocprof` hardware counters (HBM read/write BW, L2 hit rate, CU occupancy, Matrix Core utilization), `omniperf` roofline analysis, and firmware telemetry (HBM controller queue depth, thermal state) captured through a custom wrapper. Ran BERT-Large (seq 512, batch 16→64), DLRM (embedding-heavy), and ResNet50 through the harness. Classified every kernel into compute-bound / memory-bound / comms-bound using measured arithmetic intensity vs the MI200 roofline. Found that at the batch sizes customers cared about, ~55% of BERT decode-equivalent time was memory-bandwidth-bound and another ~20% was stuck in RCCL AllReduce because the model didn't fit in one GPU. Presented the data to the CDNA architecture team with the recommendation: the single biggest per-GPU throughput lever was not more FLOPS — it was enough HBM capacity to keep 70B-class models resident on one device and eliminate mandatory tensor parallelism.

Result. Data contributed to the architectural decision to ship MI300X with 192 GB HBM3 (2.4× H100's 80 GB), enabling single-GPU inference for 70B models and eliminating AllReduce overhead that had bottlenecked the prior generation. This capacity-first strategy became AMD's defining competitive moat and carried forward into MI325X (256 GB), MI355X (288 GB), and MI450 (432 GB HBM4).

What to emphasize in interviews: "I wasn't just optimizing on silicon — I was generating the quantitative evidence that shaped the silicon."

STAR #2 — GEMM Kernel Optimization → Matrix Core Design Feedback (MI200 → MI300/MI355X)

Situation. GEMM kernels in BERT, DLRM, and ResNet50 on MI200 were achieving only 55–60% of the Matrix Core's theoretical peak FP16/BF16 throughput — the ~30% gap to NVIDIA H100's cuBLAS was visible in every head-to-head benchmark.

Task. Close the GEMM efficiency gap on existing silicon, then feed the bottleneck analysis back to the Matrix Core design team to influence CDNA 3 and CDNA 4 tensor unit architecture.

Action. Profiled GEMM kernel executions with `rocprof` and Nsight-equivalent instrumentation, instrumented at the wave/SIMD level to identify tile-size mismatches, register spill pressure, and shared-memory bank conflicts. Re-tuned tile dimensions to align with the MI200 Matrix Core's native MMA shape, optimized shared memory staging to eliminate bank conflicts, improved thread scheduling to keep Matrix Cores saturated during weight load, and leveraged asynchronous copy to hide HBM latency behind compute. Worked with the compiler team on LLVM/ROCm code-generation changes to emit better MMA instruction sequences for common tile shapes. Documented which GEMM shapes remained inefficient even after tuning — specifically small-K dimensions and non-power-of-two shapes common in transformer attention — and filed these as architectural requests: wider dispatch, more flexible tile shapes, and lower-precision accumulator support (FP8 and eventually FP4/FP6).

Result. Delivered 20% GEMM kernel performance improvement on existing MI200 silicon, directly improving BERT training throughput and ResNet50 inference latency. More strategically, the shape-efficiency data contributed to CDNA 3's revised Matrix Core tile flexibility and CDNA 4's FP4/FP6 support in MI355X. The low-precision recommendation was vindicated when MI355X's FP4 capability delivered ~2× compute uplift vs MI300X's FP8-only path, matching NVIDIA Blackwell's precision strategy.

What to emphasize: You weren't just tuning kernels — you were driving the precision roadmap. FP4/FP6 in CDNA 4 is exactly the kind of feature that comes from a perf engineer saying "here's the shape of workloads we're losing on, here's why, here's the silicon feature that fixes it."

STAR #3 — MIMS Simulator → HW/SW Co-Design Flywheel (MI300 → MI355X → MI400)

Situation. The feedback loop between workload profiling on current silicon and architectural decisions for the next generation was too slow — by the time a bottleneck was confirmed on real MI200/MI300 hardware, the next-gen design (MI355X) was already past the point where

significant changes could be absorbed. The team needed a way to evaluate proposed architectural changes against real ML workloads *before* tape-out.

Task. Build and validate the Machine Intelligence Model Simulator (MIMS) — a hardware-level simulator capable of modeling transformer and CNN workloads accurately enough to drive pre-silicon architectural decisions.

Action. Co-designed MIMS with architecture, compiler, and workloads teams. Integrated BERT, DLRM, ResNet50, and eventually LLaMA-class transformer workloads as first-class benchmarks. Instrumented the simulator to emit the same performance counter taxonomy as rocprof/omniperf on real silicon, enabling apples-to-apples validation. Calibrated simulated HBM bandwidth, Matrix Core throughput, and Infinity Fabric behavior against measured MI200 and MI300 runs until simulation error was within single-digit percent for representative kernels. Used MIMS to evaluate proposed CDNA 4 changes before tape-out: FP4/FP6 Matrix Core design, reduced base-die count (4→2) to lower cross-die latency, and HBM3e bandwidth targets. Demonstrated that the 2-base-die design would reduce BERT AllReduce latency by a measurable margin and that FP4 would keep Matrix Cores saturated at inference batch sizes where FP8 was leaving throughput on the table.

Result. MIMS reduced pre-silicon architectural iteration time by ~30%, directly contributing to the architectural decisions locked into MI355X (CDNA 4): FP4/FP6 support, fewer base dies for lower intra-package latency, and HBM3e at 8 TB/s. The simulator methodology became the template for evaluating MI400-series design choices — specifically UALink fabric decisions and HBM4 capacity trade-offs.

What to emphasize: "Pre-silicon architectural influence" is the single highest-leverage thing a perf engineer can claim. If you contributed to MIMS, lead with this story for senior/staff-level roles.

STAR #4 — PCIe & System-Level Debug → Fabric/NIC Architectural Changes (MI200 → MI300 → MI400)

Situation. Multi-GPU training on MI200-based systems showed throughput scaling that fell short of linear beyond 8 GPUs. PCIe Gen4 and early Infinity Fabric configurations were suspected, but the team lacked a clean decomposition of where the nanoseconds were going — CPU-GPU transfer, inter-GPU fabric, NIC RDMA path, or firmware overhead.

Task. Build a system-level profiling capability that could decompose multi-GPU communication latency into its component hops, then use it to recommend fabric and NIC architectural changes for the next platform generation.

Action. Profiled PCIe data transfers using Linux `perf`, custom RDMA tracepoints, and vendor firmware counters. Ran controlled experiments varying packet size, queue depth, and

congestion patterns across BERT multi-node training and DLRM embedding lookup workloads. Identified three distinct bottlenecks: PCIe Gen4 saturation during optimizer-state AllGather in FSDP-style training, firmware queue stalls under bursty DLRM embedding traffic, and CPU proxy-thread contention during RCCL collectives. Tuned firmware queuing parameters and driver configuration to reduce observed latency by ~15%. More importantly, filed detailed recommendations to the platform team: PCIe Gen5 minimum for next gen, dedicated firmware paths for collective traffic, and — at the fabric level — a move toward a more capable, open interconnect to compete with NVLink.

Result. 15% PCIe data-transfer performance improvement on existing platforms. Longer-term, the system-level data contributed to AMD's platform roadmap decisions that appeared in MI300-series servers (PCIe Gen5) and MI400-series (UALink open interconnect standard as AMD's answer to NVLink). The RCCL / proxy-thread work mirrors what Meta later published at SIGCOMM 2024 for their RoCE CTS congestion control — you were solving the same class of problem in-house at AMD a generation earlier.

What to emphasize: The SIGCOMM 2024 Meta RoCE paper is an open-source receipt that this class of work matters. You can say "the Meta RoCE CTS work at SIGCOMM is the same category of problem I was solving at AMD at the platform level — receiver-driven admission, firmware proxy optimization, collective-aware congestion control."

STAR #5 — CAC & Power-Perf Co-Optimization → TDP Envelope for MI300X/MI355X

Situation. The jump from MI200 (~500W) to MI300X (750W) to MI355X (1,400W) represented a 3× TDP increase in two generations. Without careful power-perf co-design, the higher-TDP parts would hit thermal throttling on real workloads and lose the advertised throughput gains — a repeat of the industry pattern where paper TFLOPS don't survive contact with production data centers.

Task. Characterize dynamic CAC (Active Capacitance) and thermal behavior under real ML workloads, and recommend clock/voltage and power-management strategies that balanced sustained performance against thermal stability for the next-gen TDP envelope.

Action. Instrumented MI200 and early MI300 silicon with per-block CAC telemetry under BERT training, DLRM inference, and synthetic GEMM stress. Mapped the activity factor of compute units, HBM controllers, and Infinity Fabric at workload phase granularity (prefill vs decode, embedding lookup vs MLP, forward vs backward). Identified that certain workload phases spiked CAC well above thermal design assumptions, driving aggressive frequency throttling and losing sustained throughput. Tested fine-tuned clock configurations and adaptive DVFS policies that anticipated workload phase transitions rather than reacting to thermal sensors. Fed the CAC-per-workload data into the physical-design and power-management teams for CDNA 4 so the

MI355X TDP envelope and boost behavior could be sized against realistic ML activity factors, not worst-case synthetic peaks.

Result. 10% power reduction on existing silicon at matched performance, validated across BERT, DLRM, and ResNet50. Downstream, the CAC-per-workload methodology informed the MI355X 1,400W TDP envelope design and the liquid-cooling mandate — ensuring that advertised peak throughput is actually sustainable under production ML workloads rather than thermally throttled. Directly relevant to MSFT/Meta TCO conversations: power is the #1 data center constraint, and this work is exactly the kind of data that feeds into rack-level power budgeting.

What to emphasize: "I was the person who made sure our TDP numbers survived contact with real workloads." Power/thermal co-design is increasingly an interview topic because of the 1.4kW-per-GPU reality — you've done it.

STAR #6 — HPC DGEMM → Training Precision Strategy (MI200 → MI300 → MI355X)

Situation. HPC customers running DGEMM (FP64) on MI200 for scientific workloads were not hitting the performance parity with NVIDIA A100/H100 that marketing had promised, hurting AMD's competitiveness in the exascale HPC market (a key reference customer base ahead of the AI push).

Task. Optimize DGEMM on MI200/MI250X and use the findings to shape FP64 and broader precision strategy across CDNA 3 and CDNA 4.

Action. Analyzed existing rocBLAS DGEMM implementations with `omniperf` and `rocprof`, identifying that critical kernel sections were leaving Matrix Core FP64 throughput unrealized due to sub-optimal memory layout and insufficient latency hiding. Re-engineered the algorithm's blocking strategy, leveraged hardware accelerator paths more aggressively, and optimized memory layouts for the MI200 cache hierarchy. Iterative validation against reference LINPACK and production HPC codes (GROMACS, LAMMPS). Parallel to this, documented the observation that HPC FP64 demand was stable while the AI market was pulling rapidly toward lower precision — recommending the architecture team preserve FP64 throughput but invest aggressive die area into BF16/FP8/FP4 as the growth vector.

Result. 25% DGEMM throughput improvement on MI200-class hardware, strengthening AMD's HPC competitiveness and securing exascale wins (Frontier / El Capitan lineage). The precision-strategy recommendation — hold FP64 steady, invest in sub-byte formats — mapped directly onto CDNA 4's FP4/FP6 addition while preserving FP64 for HPC, a balance MI355X explicitly hits at ~80 TFLOPS FP64 and ~5 PFLOPS FP8.

Whenever an interviewer asks "what's the most impactful thing you built," lead with this:

> *"The highest-leverage thing I built at AMD wasn't any single optimization — it was a unified workload-profiling methodology that combined ROCm hardware counters, firmware telemetry, and OS-level perf traces into a single roofline-based bottleneck classification across BERT, DLRM, and ResNet50. That methodology turned every customer escalation into structured data that fed architectural decisions for the next generation. It's the same pattern Meta later published as Zoomer — fleet-wide automated bottleneck identification driving one-line fixes and silicon feedback — and it's what I'd build first at Microsoft or OpenAI to close the gap between workload pain and roadmap decisions."*

This is the single most senior-sounding claim you can make, because it reframes you from "kernel optimizer" to "systems engineer who drives silicon."

Tools & Techniques Vocabulary to Deploy

Layer	Tool	What you claim
Kernel counters	rocprof, omniperf	Per-kernel HBM BW, L2 hit rate, Matrix Core utilization, wave occupancy
System timeline	ROCm System Profiler, Linux perf	CPU↔GPU sync gaps, kernel launch overhead, RCCL stalls
Memory BW	rocm-bandwidth-test, STREAM triad	Peak vs sustained HBM roofline characterization
Firmware	Custom telemetry hooks	CAC, thermal, queue depth, voltage/frequency state
Roofline	omniperf + custom scripts	Arithmetic-intensity-based bottleneck classification
Pre-silicon	MIMS simulator	Workload-driven architectural evaluation before tape-out
Communication	RCCL tests, ib_read_bw equiv	Collective BW, AllReduce latency at scale
Regression	CI-based perf harness	Catch regressions across ROCm releases, kernel changes

Map each to the NVIDIA equivalent when answering MSFT/OpenAI interviews: rocprof ↔ Nsight Compute, omniperf ↔ Nsight Compute roofline, ROCm System Profiler ↔ Nsight Systems, RCCL ↔ NCCL. This shows you're platform-fluent.

Generational Bottleneck-to-Architecture Map (Reference)

Generation	Dominant bottleneck you measured	Architectural recommendation	Next-gen feature that shipped
MI100 → MI200	FP16/BF16 Matrix Core utilization	Wider MMA, better dispatch	CDNA 2 Matrix Core v2, dual-GCD
MI200 → MI300	HBM capacity forcing TP / AllReduce	Massively more HBM per GPU	MI300X 192 GB HBM3, chiplet stacking
MI300 → MI355X	FP8 compute saturating before memory; cross-die latency	Sub-byte precision; fewer base dies	CDNA 4 FP4/FP6; 2 base dies; 288 GB HBM3e
MI355X → MI400	Intra-node fabric bandwidth vs NVLink 5	Open high-BW interconnect	UALink; HBM4; Helios rack

Use this table in interviews to show you can narrate the entire CDNA arc as a cause-and-effect between workload profiling and architectural response.